

RA1. Aplicaciones compuestas por varios procesos (programación paralela)

1. Análisis de procesos y concurrencia

Pregunta 1

Observa el siguiente código en Java:

```
try {  
    Process p = Runtime.getRuntime().exec("notepad.exe");  
    p.waitFor();  
    System.out.println("El proceso ha terminado");  
} catch (Exception e) {  
    e.printStackTrace();  
}
```

¿Qué ocurrirá cuando se ejecute este código?

- a) Se ejecutará Notepad y el programa Java continuará inmediatamente.
- b) Se ejecutará Notepad y el programa Java esperará a que Notepad se cierre.
- c) El código lanza una excepción por uso incorrecto de `waitFor()`.
- d) Se ejecutará Notepad, pero el mensaje "El proceso ha terminado" se mostrará antes de abrirse la aplicación.

 Respuesta correcta: b)

Pregunta 2

¿Cuál es la **diferencia principal** entre programación **paralela** y **distribuida**?

- a) La paralela usa varios equipos y la distribuida usa un solo procesador.
- b) La paralela utiliza múltiples núcleos o procesadores en un mismo sistema; la distribuida usa varios equipos conectados en red.
- c) Ambas son lo mismo.
- d) La distribuida siempre es más rápida que la paralela.

 Respuesta correcta: b)

Pregunta 3

En un sistema operativo, un **proceso** se puede definir como:

- a) Un bloque de código fuente sin ejecutar.
- b) Una unidad mínima de CPU dentro de un hilo.
- c) Un programa en ejecución con su propio espacio de memoria y recursos.
- d) Una variable global accesible por varios hilos.

 Respuesta correcta: c)

Pregunta 4

Observa el siguiente código:

```
ProcessBuilder pb = new ProcessBuilder("calc.exe");  
Process p = pb.start();  
System.out.println("PID: " + p.pid());
```

¿Qué imprime este código?

- a) El identificador del proceso que ejecuta Java.
- b) El identificador del proceso de la calculadora.
- c) Un número aleatorio.
- d) Nada, ya que `pid()` no existe en la clase `Process`.

 Respuesta correcta: b)

Pregunta 5

¿Qué método permite **leer la salida** de un proceso iniciado desde Java?

- a) `getInputStream()`
- b) `readOutput()`
- c) `outputStream()`
- d) `getProcessOutput()`

 Respuesta correcta: a)

Pregunta 6

¿Qué sucede si se crean varios procesos desde Java sin sincronizarlos correctamente?

- a) Se ejecutan en orden predecible.
- b) Pueden competir por recursos, causando errores o bloqueos.
- c) El sistema operativo los cancela automáticamente.
- d) Todos comparten el mismo espacio de memoria sin problemas.

 Respuesta correcta: b)

1. Creación y ejecución de hilos

Pregunta 7

Analiza el código siguiente:

```
class MiHilo extends Thread {  
    public void run() {  
        System.out.println("Ejecutando hilo");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        MiHilo h = new MiHilo();  
        h.start();  
        System.out.println("Fin del main");  
    }  
}
```

¿Cuál podría ser la salida?

a)

Ejecutando hilo
Fin del main

b)

Fin del main
Ejecutando hilo

c)

Depende del planificador del sistema operativo.

d)

Lanza una excepción al ejecutar `start()`.

✓ Respuesta correcta: c)

Pregunta 8

¿Qué diferencia existe entre llamar a `hilo.run()` y `hilo.start()`?

- a) Ninguna, ambos inician el hilo.
- b) `run()` crea un nuevo hilo; `start()` ejecuta en el mismo hilo principal.
- c) `start()` crea un nuevo hilo y llama internamente a `run()`, mientras que `run()` solo ejecuta el código en el hilo actual.
- d) `start()` solo está disponible en clases que implementan `Runnable`.

✓ Respuesta correcta: c)

Pregunta 9

Observa este código:

```
class Contador implements Runnable {  
    private int valor = 0;  
    public void run() {  
        for(int i=0; i<1000; i++) valor++;  
    }  
}
```

Si se ejecutan **dos hilos** con la misma instancia de `Contador`, ¿qué problema puede surgir?

- a) Ninguno, los hilos son independientes.
- b) Puede haber condiciones de carrera y resultados inconsistentes.
- c) Los hilos no comparten memoria.
- d) Los hilos se bloquean automáticamente.

✓ Respuesta correcta: b)

2. Sincronización y comunicación

Pregunta 10

¿Cómo se puede garantizar que solo un hilo acceda a un bloque de código a la vez?

- a) Usando `volatile`.
- b) Usando `synchronized`.
- c) Usando `final`.
- d) No es posible hacerlo en Java.

✓ Respuesta correcta: b)

Pregunta 11

¿Qué hace el método `join()` en un hilo?

- a) Duplica el hilo actual.
- b) Espera a que otro hilo termine antes de continuar.
- c) Sincroniza variables entre hilos.
- d) Finaliza el hilo inmediatamente.

✓ Respuesta correcta: b)

Pregunta 12

¿Qué indica el método `setPriority(Thread.MAX_PRIORITY)`?

- a) Asigna una prioridad máxima al hilo, aunque el sistema operativo puede ignorarla.
- b) Obliga al sistema a ejecutar primero ese hilo.
- c) Finaliza los hilos con menor prioridad.
- d) Cambia el tipo de ejecución a distribuida.

✓ Respuesta correcta: a)

3. Librerías específicas y gestión avanzada

Pregunta 13

¿Qué ventaja tiene usar un **ExecutorService** frente a crear hilos manualmente?

- a) Permite reutilizar hilos y gestionar un pool de ejecución.
- b) Crea procesos en lugar de hilos.
- c) Ejecuta código más rápido por defecto.
- d) No permite sincronización.

✓ Respuesta correcta: a)

Pregunta 14

¿Qué problema resuelve la clase `Future<T>`?

- a) Evita fugas de memoria.
- b) Permite obtener el resultado de un hilo asíncrono cuando esté disponible.
- c) Aumenta el rendimiento de los hilos.
- d) Controla excepciones automáticamente.

✅ Respuesta correcta: b)

Pregunta 15

Analiza este código:

```
ExecutorService executor = Executors.newFixedThreadPool(2);
executor.submit(() -> System.out.println("Tarea 1"));
executor.submit(() -> System.out.println("Tarea 2"));
executor.shutdown();
```

¿Qué sucede si se añaden más tareas después del `shutdown()`?

- a) Se ejecutarán todas igualmente.
- b) Generará una excepción `RejectedExecutionException`.
- c) Las tareas quedarán en espera.
- d) El programa las ignora silenciosamente.

✅ Respuesta correcta: b)

Perfecto 🍷 te dejo una **nueva tanda de preguntas tipo test** centradas **exclusivamente en los hilos de ejecución en Java**, su **creación, métodos, estados, sincronización y comunicación**, todas con **análisis de código realista**.

Estas preguntas cubren los criterios del **RA2**: creación, control, estados, sincronización, prioridad, y librerías específicas.



Bloque 1: Creación y ejecución de hilos

Pregunta 1

Analiza el siguiente código:

```
class MiHilo extends Thread {
    public void run() {
        System.out.println("Hilo en ejecución");
    }
}

public class Prueba {
    public static void main(String[] args) {
        MiHilo h = new MiHilo();
        h.run();
        System.out.println("Fin del main");
    }
}
```

¿Qué sucede al ejecutar este código?

- a) El hilo se ejecuta en paralelo con el `main`.
- b) Se lanza una excepción al no usar `start()`.
- c) El método `run()` se ejecuta dentro del hilo principal, no en paralelo.
- d) No se muestra ningún mensaje.

 Respuesta correcta: c)

Pregunta 2

Dado el siguiente código:

```
class Demo extends Thread {
    public void run() {
        System.out.println(Thread.currentThread().getName());
    }
}

public class Main {
    public static void main(String[] args) {
        Demo h1 = new Demo();
        h1.setName("Hilo A");
        h1.start();
        System.out.println(Thread.currentThread().getName());
    }
}
```

}

¿Qué se mostrará por pantalla?

- a) Siempre primero “main” y luego “Hilo A”.
- b) Siempre primero “Hilo A” y luego “main”.
- c) El orden es impredecible.
- d) Solo se mostrará “Hilo A”.

✓ Respuesta correcta: c)

Pregunta 3

¿Qué sucede si intentamos ejecutar `hilo.start()` dos veces sobre el mismo objeto `Thread`?

- a) El hilo se ejecuta dos veces.
- b) Se crea una copia del hilo.
- c) Se lanza una excepción `IllegalThreadStateException`.
- d) No pasa nada, se ignora la segunda llamada.

✓ Respuesta correcta: c)

Bloque 2: Estados y control de ejecución

Pregunta 4

¿Cuál de las siguientes llamadas **pone un hilo en espera temporalmente** sin liberar su bloqueo?

- a) `sleep(1000)`
- b) `wait()`
- c) `yield()`
- d) `join()`

✓ Respuesta correcta: a)

Pregunta 5

¿Qué hace el método `yield()`?

- a) Detiene el hilo actual.
- b) Hace que el hilo actual ceda voluntariamente la CPU a otro hilo de igual prioridad.
- c) Suspende el hilo hasta que otro lo notifique.
- d) Bloquea el hilo actual de forma indefinida.

✓ Respuesta correcta: b)

Pregunta 6

¿Cuál de los siguientes métodos puede lanzar `InterruptedException`?

- a) `start()`
- b) `sleep()`
- c) `setPriority()`
- d) `isAlive()`

✓ Respuesta correcta: b)

Pregunta 7

Dado el siguiente código:

```
Thread t = new Thread(() -> {
    try {
        Thread.sleep(500);
        System.out.println("Hilo despertado");
    } catch (InterruptedException e) {
        System.out.println("Hilo interrumpido");
    }
});

t.start();
t.interrupt();
```

¿Qué salida es posible?

- a) Siempre "Hilo despertado".
- b) Siempre "Hilo interrumpido".
- c) Podría ser cualquiera de las dos, según el momento de la interrupción.
- d) Ninguna, lanza excepción sin capturar.

✓ Respuesta correcta: c)

Pregunta 8

¿Qué devuelve el método `isAlive()` de un hilo?

- a) `true` si el hilo ha sido creado, aunque no haya empezado.
- b) `true` mientras el hilo se está ejecutando o está en espera.
- c) `true` solo si el hilo está en estado `RUNNABLE`.
- d) Siempre devuelve `false` cuando el hilo está dormido.

✓ Respuesta correcta: b)



Bloque 3: Sincronización y comunicación entre hilos

Pregunta 9

¿Qué hace el siguiente código?

```
synchronized(obj) {  
    obj.wait();  
}
```

- a) Suspende el hilo hasta que otro hilo invoque `notify()` o `notifyAll()` sobre el mismo objeto.
- b) Suspende el hilo para siempre.
- c) Lanza una excepción porque `wait()` no puede usarse con `synchronized`.
- d) Crea un nuevo hilo de espera.

✓ Respuesta correcta: a)

Pregunta 10

En Java, ¿qué métodos deben llamarse **dentro de un bloque sincronizado** para evitar `IllegalMonitorStateException`?

- a) `sleep()` y `join()`
- b) `notify()`, `notifyAll()` y `wait()`
- c) `interrupt()` y `isAlive()`
- d) `start()` y `stop()`

✓ Respuesta correcta: b)

Pregunta 11

Analiza el siguiente código:

```
class Contador {  
    private int c = 0;  
    public synchronized void incrementar() {  
        c++;  
    }  
    public int getValor() { return c; }  
}
```

¿Qué garantiza el modificador `synchronized` en `incrementar()`?

- a) Que ningún otro hilo podrá acceder al objeto `Contador`.
- b) Que solo un hilo puede ejecutar ese método a la vez.
- c) Que `getValor()` también estará sincronizado.
- d) Que el método se ejecutará más rápido.

✓ Respuesta correcta: b)

Pregunta 12

¿Qué sucede si dos hilos intentan acceder simultáneamente a un bloque sincronizado sobre **el mismo objeto monitor**?

- a) Ambos hilos se ejecutan a la vez.
- b) Uno de ellos espera hasta que el otro libere el monitor.
- c) Se lanza una excepción.
- d) El sistema operativo decide cuál ejecuta.

✓ Respuesta correcta: b)

Bloque 4: Prioridad y control avanzado

Pregunta 13

¿Qué rango de prioridades se puede asignar a un hilo en Java?

- a) 0 a 5
- b) 1 a 10
- c) -10 a 10
- d) 0 a 255

✓ Respuesta correcta: b)

Pregunta 14

Dado el código:

```
Thread t1 = new Thread(() -> System.out.println("A"));
t1.setPriority(Thread.MAX_PRIORITY);
Thread t2 = new Thread(() -> System.out.println("B"));
t2.setPriority(Thread.MIN_PRIORITY);
t1.start();
t2.start();
```

¿Qué salida se garantiza?

- a) Siempre "A" antes que "B".
- b) Siempre "B" antes que "A".
- c) El orden es no determinista; la prioridad es solo una sugerencia al planificador.
- d) Lanza excepción por conflicto de prioridades.

 Respuesta correcta: c)

Pregunta 15

¿Cuál es la función principal del método `join()`?

- a) Pausar el hilo actual hasta que otro hilo termine.
- b) Ejecutar dos hilos al mismo tiempo.
- c) Reiniciar un hilo detenido.
- d) Fusionar los resultados de dos hilos.

 Respuesta correcta: a)

Pregunta 16

Analiza el siguiente código:

```
Thread t1 = new Thread(() -> {
    for(int i=0; i<3; i++) {
        System.out.println("Tarea " + i);
    }
});
t1.start();
t1.join();
System.out.println("Fin del programa");
```

¿Qué garantiza `t1.join()`?

- a) Que el `main` esperará a que `t1` termine antes de imprimir “Fin del programa”.
- b) Que el hilo `t1` se ejecutará indefinidamente.
- c) Que ambos hilos se ejecuten en paralelo.
- d) Nada, `join()` no afecta al flujo del `main`.

✓ Respuesta correcta: a)

Bloque 5: Ejecutor y librerías de concurrencia

Pregunta 17

¿Cuál de las siguientes clases forma parte del paquete `java.util.concurrent`?

- a) `ThreadPool`
- b) `ExecutorService`
- c) `ThreadHandler`
- d) `RunnableExecutor`

✓ Respuesta correcta: b)

Pregunta 18

¿Qué ventaja tiene usar `Callable` en lugar de `Runnable`?

- a) `Callable` permite devolver un valor y lanzar excepciones.
- b) `Callable` se ejecuta más rápido.
- c) `Runnable` no puede ser usado con `ExecutorService`.
- d) `Callable` no necesita implementarse en una clase.

✓ Respuesta correcta: a)

Pregunta 19

Dado el siguiente código:

```
ExecutorService executor = Executors.newSingleThreadExecutor();
Future<Integer> resultado = executor.submit(() -> 5 + 10);
System.out.println(resultado.get());
executor.shutdown();
```

¿Qué mostrará por pantalla?

- a) 5 + 10
- b) 15
- c) Nada, ya que `get()` no devuelve nada.
- d) Excepción por uso de `Future`.

✓ Respuesta correcta: b)

Pregunta 20

¿Qué sucede si no se llama a `shutdown()` en un `ExecutorService`?

- a) Los hilos terminan automáticamente al acabar las tareas.
- b) El programa puede quedar en ejecución sin finalizar.
- c) Lanza una excepción al terminar.
- d) No afecta en absoluto.

✓ Respuesta correcta: b)